Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 5

«API и BLoC (часть 2)»

По дисциплине “Кроссплатформенные средства разработки”

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ковалев С.В.

Чебоксары, 2026

Цель: закрепить навыки работы с API, HTTP-запросами и управления состоянием Flutter-приложения с помощью архитектуры BLoC.

https://github.com/kukirmash/kukirmash_weather_app

Ход работы:

1) Запрос подробностей дня по идентификатору

Второй экран должен получать данные конкретного дня. Для этого используется тот же endpoint Open-Meteo, но запрос ограничивается одной датой через параметры start_date и end_date. Дополнительно запрашивается почасовой блок данных.

```
GET https://api.open-meteo.com/v1/forecast
  ?latitude=56.13
  &longitude=47.25
  &timezone=auto
  &daily=weather_code,temperature_2m_max,...
  &hourly=temperature_2m,relative_humidity_2m,
          apparent_temperature,precipitation_probability,
          weather_code,wind_speed_10m,surface_pressure
  &start_date=2026-10-19
  &end_date=2026-10-19
```

2) Модели подробностей и почасового прогноза

Результат второго запроса представлен классом DayDetails, который хранит суточную сводку и список почасовых значений. Каждый час описан моделью HourlyForecast: время, температура, влажность, ощущаемая температура, вероятность осадков, код погоды, ветер и давление.

lib/data/forecast/day_details.dart

```
class DayDetails extends Equatable {
  const DayDetails({required this.summary, required this.hourly});

  final DailyForecast summary;
  final List<HourlyForecast> hourly;

  @override
  List<Object?> get props => [summary, hourly];
}
```

lib/data/forecast/hourly_forecast.dart (фрагмент)

```
class HourlyForecast extends Equatable {
  const HourlyForecast({
    required this.time,
    required this.temperature,
    required this.relativeHumidity,
    required this.apparentTemperature,
    required this.precipitationProbability,
    required this.weatherCode,
    required this.windSpeed,
    required this.surfacePressure,
```

```
  });

  final DateTime time;
  final double temperature;
  final int relativeHumidity;
  final double apparentTemperature;
  final int precipitationProbability;
  final int weatherCode;
  final double windSpeed;
  final double surfacePressure;

  String get formattedTime =>
      '${time.hour.toString().padLeft(2, '0')}:00';
}
```

3) Получение данных в репозитории

Метод getDayDetails проверяет корректность даты, формирует запрос на один день, разбирает ответ и возвращает объект DayDetails. Если сервер не прислал суточную сводку, выбрасывается ForecastException с понятным сообщением.

lib/data/forecast/forecast_repository.dart (фрагмент)

```
@override
Future<DayDetails> getDayDetails(String id) async {
  final date = DateTime.tryParse(id);
  if (date == null) {
    throw const ForecastException('Некорректная дата прогноза.');
  }

  final isoDate = date.toIso8601String().split('T').first;

  try {
    final response = await dio.get<Map<String, dynamic>>(
      Endpoints.forecast,
      queryParameters: {
        'daily': _dailyParameters,
        'hourly': _hourlyParameters,
        'start_date': isoDate,
        'end_date': isoDate,
      },
    );

    final parsed = ForecastResponse.fromJson(response.data!);
    final daily = parsed.toDailyForecasts();
    if (daily.isEmpty) {
      throw const ForecastException('Нет данных за выбранный день.');
    }

    return DayDetails(
      summary: daily.first,
      hourly: parsed.toHourlyForecasts(),
    );
  } on DioException catch (exception) {
    throw ForecastException(_dioMessage(exception));
  }
}
```

4) BLoC второго экрана

Для второго экрана создан DetailsBloc. Событие DetailsLoadRequested передаёт идентификатор дня и необязательный Completer для обновления. Состояния DetailsInitial, DetailsLoading, DetailsLoaded и DetailsError

описывают все режимы экрана. При ошибке обновления уже загруженные данные сохраняются, а пользователю показывается уведомление.

lib/app/features/details/bloc/details_event.dart

```
class DetailsLoadRequested extends DetailsEvent {
  const DetailsLoadRequested({required this.id, this.completer});

  final String id;
  final Completer<void>? completer;

  @override
  List<Object?> get props => [id];
}
```

lib/app/features/details/bloc/details_state.dart

```
final class DetailsLoading extends DetailsState {
  const DetailsLoading();
}

final class DetailsLoaded extends DetailsState {
  const DetailsLoaded({required this.details, this.refreshError});

  final DayDetails details;
  final Object? refreshError;
}

final class DetailsError extends DetailsState {
  const DetailsError({required this.exception});
  final Object exception;
}
```

lib/app/features/details/bloc/details_bloc.dart

```
class DetailsBloc extends Bloc<DetailsEvent, DetailsState> {
  DetailsBloc(this._forecastRepository) : super(DetailsInitial()) {
    on<DetailsLoadRequested>(_onLoadRequested);
  }

  final ForecastRepositoryInterface _forecastRepository;

  Future<void> _onLoadRequested(
    DetailsLoadRequested event,
    Emitter<DetailsState> emit,
  ) async {
    final previous = state;
    final loaded =
        previous is DetailsLoaded && previous.details.summary.id == event.id
        ? previous
        : null;

    if (loaded == null) emit(const DetailsLoading());

    try {
      final details = await _forecastRepository.getDayDetails(event.id);
      if (isClosed) return;
      emit(DetailsLoaded(details: details));
    } catch (exception, stackTrace) {
      talker.handle(exception, stackTrace);
      if (isClosed) return;
      if (loaded != null) {
        emit(DetailsLoaded(
          details: loaded.details,
          refreshError: exception,
        ));
      } else {
        emit(DetailsError(exception: exception));
      }
    } finally {
      event.completer?.complete();
    }
  }
}
```

5) Интеграция BLoC с экраном

Экран создаёт DetailsBloc через BlocProvider и сразу отправляет событие загрузки. BlocBuilder выбирает нужное представление в зависимости от состояния. Для обновления используется RefreshIndicator.

lib/app/features/details/details_screen.dart (фрагмент)

```
BlocBuilder<DetailsBloc, DetailsState>(
  builder: (context, state) {
    return switch (state) {
      DetailsInitial() || DetailsLoading() => const AppLoading(
        message: 'Загружаем подробности',
      ),
      DetailsError(:final exception) => AppError(
        message: exception.toString(),
        onRetry: () => context.read<DetailsBloc>().add(
          DetailsLoadRequested(id: id),
        ),
      ),
      DetailsLoaded(:final details, :final refreshError) =>
        _DetailsContent(
          details: details,
          refreshError: refreshError,
        ),
    };
  },
)
```

6) Почасовой прогноз и описание дня

На экране деталей данные выводятся в удобном виде: сетка основных показателей, горизонтальная лента почасового прогноза и карточка описания с рекомендацией. Виджет HourlyForecastTile подсвечивает текущий час и показывает вероятность осадков.

lib/app/features/details/widgets/hourly_forecast_tile.dart

```
class HourlyForecastTile extends StatelessWidget {
  const HourlyForecastTile({
    super.key,
    required this.forecast,
    this.highlighted = false,
  });

  final HourlyForecast forecast;
  final bool highlighted;

  @override
  Widget build(BuildContext context) {
    return Container(
      width: 86,
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 12),
      decoration: BoxDecoration(
        color: highlighted
            ? ThemeColors.primary.withValues(alpha: 0.08)
            : ThemeColors.surface,
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: ThemeColors.line),
      ),
      child: Column(
        children: [
          Text(forecast.formattedTime),
          Image.asset(forecast.imagePath, width: 38, height: 38),
```

```
          Text('${forecast.temperature.round()}°'),
          Text('${forecast.precipitationProbability}%'),
        ],
      ),
    );
  }
}
```

7) Результат

Второй экран получает подробности выбранного дня по его идентификатору, отображает почасовой прогноз и корректно обрабатывает загрузку, ошибки и повторное обновление. На рисунках показан итоговый вид экрана.

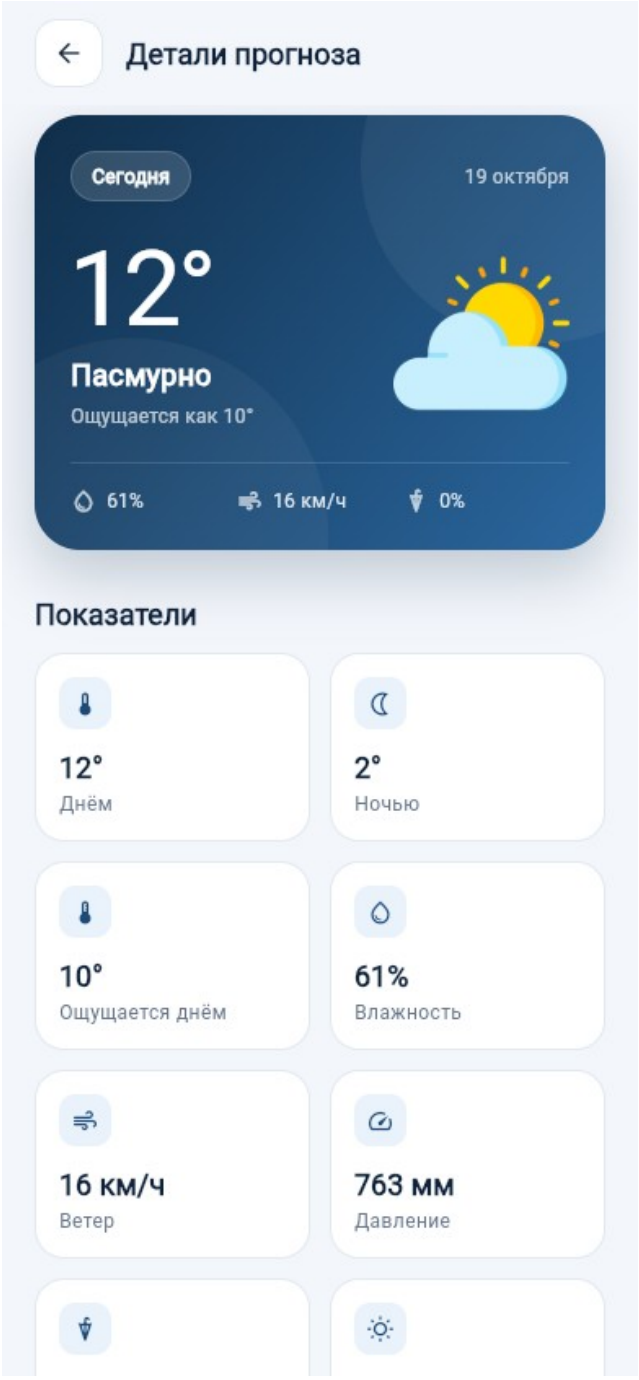


Рисунок 1 — экран подробностей дня



Рисунок 2 — почасовой прогноз и описание

Проект проверен командами flutter analyze и flutter test: анализатор не обнаружил замечаний, все 13 тестов успешно проходят.

Вывод:

в ходе работы был реализован запрос подробностей дня по идентификатору, создан DetailsBloc, выполнена интеграция второго экрана с API, добавлены почасовой прогноз и обработка ошибок обновления.